

FABRICPERF: Measuring NIC-less Scale-Up Network through GPU Communication Kernel Profiling

Songlin Huang*
University of Hong Kong
Hong Kong SAR
huangs0@connect.hku.hk

Chenshu Wu
University of Hong Kong
Hong Kong SAR
chenshu@cs.hku.hk

Abstract

The rapid growth of distributed machine learning systems shifts ML networks from the CPU/GPU-NIC to GPU-Fabrics architectures, creating an emerging need to measure and optimize NIC-less Scale-up networks. Yet, the disappearance of NICs removes key observability points and leaves existing measurement tools largely limited to coarse, application-layer telemetry. We present FABRICPERF, a Scale-up network measurement toolkit that enables fine-grained packet-level timestamping and cross-layer hardware insights. FABRICPERF uses a software-hardware co-design that integrates protocol-level probing and GTPP clock synchronization within GPU communication kernels with memory profiling of physical pipelines. We implement FABRICPERF for the CUDA/NvLink ecosystem and validate its high precision and low overhead via extensive experiments on H100 and GB200 NVL72. Using FABRICPERF, we explore Scale-up network dynamics under varied configurations, with two case studies that reveal critical bottlenecks: (i) channel imbalance in high P99 latency and (ii) memory pipeline inefficiency of ~0% LLC hit-rate. To mitigate these issues, we leverage work-stealing to rebalance channels by 46% and boost throughput by 17.5GB/s, and optimize LLC eviction priorities for memory flows to elevate the LLC read hit-rate to ~35%, yielding 16.07GB/s bandwidth improvement. We believe FABRICPERF improves Scale-up network observability, laying a foundation for more complete and efficient network stacks. FABRICPERF is open-sourced at github.com/open-neutrino/fabricperf.

CCS Concepts

• Networks → Network measurement; Network protocols.

Keywords

Network Measurement, Scale-Up Network, GPU Communication Kernels, GPU Kernel Profiling

ACM Reference Format:

Songlin Huang and Chenshu Wu. 2026. FABRICPERF: Measuring NIC-less Scale-Up Network through GPU Communication Kernel Profiling. In *ACM SIGCOMM 2026 Conference (SIGCOMM '26)*, August 17–21, 2026, Denver, CO, USA. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3789240.3829172>

*GB200 NVL72 experiments were conducted at Netpreme Corporation.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGCOMM '26*, Denver, CO, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2467-1/26/08

<https://doi.org/10.1145/3789240.3829172>

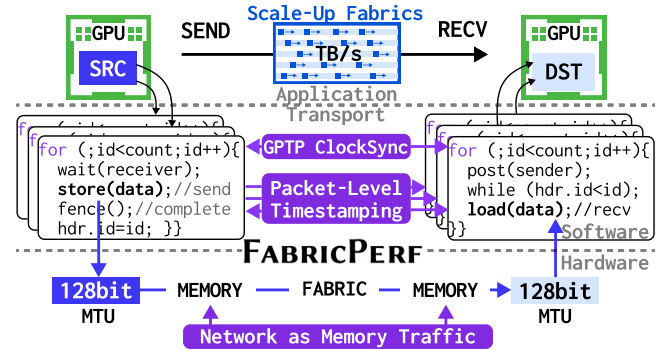


Figure 1: FABRICPERF Overview. FABRICPERF provides *fine-grained, cross-layer, and multi-faceted* Scale-up network measurements without the NIC support via three distinct designs (purple): GTPP clock synchronization, packet-level timestamping, and Network-as-Memory-Traffic modeling.

1 Introduction

The growing demand to train and serve neural networks with billions or even trillions of parameters has driven distributed machine learning (ML) systems that rely on efficient ML networks. Compared with mature data-center or HPC networks, ML network workloads exhibit distinctive characteristics, *e.g.*, large and bursty flows, collective (rather than independent) communication, in-network computation, and stricter latency/throughput requirements. These unique characteristics open new research opportunities in re-measuring and redesigning conventional protocols and infrastructures, such as congestion control [18, 55], load-balancing [37, 53, 54, 66].

Particularly, these workload characteristics have fundamentally shifted the ML networking paradigm from the traditional CPU-PCIe-NIC architectures to GPU-PCIe-NIC (*Scale-out*), and, more recently, to emerging GPU-Fabrics (*Scale-up*) networks that connect 8-72 GPUs using high-speed fabrics (*e.g.*, NvLink, UALink, Infinite-Fabrics) for TB/s bandwidth and sub-10 μ s latency. A key reason for this shift is that NIC and PCIe have become bottlenecks for high-bandwidth (TB/s/GPU), low-latency communication [6, 29].

However, the disappearance of NICs in Scale-up networks complicates both measuring network behavior and building corresponding network stacks, two mutually dependent tasks that are tightly coupled with NICs in the current network paradigm. For example, packet queues of NICs expose precise arrival timing that can inform congestion control, which in turn provides additional observability points for fine-grained measurements. In contrast, the lack of comprehensive measurement support has left Scale-up networks largely

Table 1: FABRICPERF vs. Existing Network Measurement Tools. FABRICPERF is the first *fine-grained* and *multi-faceted* Scale-up network measurement tool without NIC support and offers transport-layer (L3) latency and throughput measurements.

Category	Tool	Scope				Layer				Telemetry			
		Host	NIC	Switch	GPU	Physics	Network	Transport	Application	Latency	Throughput	Packet rate	Memory
Measurement Tools	FABRICPERF	-	-	-	Packet	✓	-	✓	-	L3, Segment	✓	✓	✓
	eBPF [14]	✓	-	-	-	-	✓	✓	-	✓	-	✓	-
	Swift [30]	✓	✓	✓	-	-	✓	✓	-	L3, Segment	-	✓	-
	hostCC [2]	✓	-	-	-	✓	-	✓	-	L3	-	-	✓
Benchmark Suites	nccl-tests [48]	✓	-	-	Kernel	-	-	-	✓	L4	L4	-	-
	Perftest [13]	✓	✓	-	-	-	-	✓	✓	L3	✓	-	-

underexplored, with their network stack remaining incomplete relative to the well-established NIC-oriented stack in Scale-out and datacenter networks (ECMP [23], DCQCN [67], PFC [26]).

Therefore, this paper aims to enable fine-grained measurements for Scale-up networks, which, however, is a challenging goal. Beyond the lack of NICs, additional challenges arise because: ① Scale-up networks run solely on GPUs in a distinct SIMT program model and are transparent to CPU/OS, which limits using or adopting host-side measurement techniques, e.g., eBPF [14]. ② The underlying fabrics/switches are often proprietary and expose limited public interfaces, and are generally less observable and programmable than NICs [21, 49], which constrains fabric-side telemetry. ③ Scale-up networks use memory semantics (ld/st), which significantly differ from de-facto send/recv and RDMA put/get in terms of MTU, leading to a substantially different network logic to measure.

These challenges limit existing software measurement tools [48] to only monitoring communication-kernel duration (as the end-to-end latency) and approximating throughput via algorithm conversions [51, 65]. Meanwhile, hardware-side tools [43] can only count total bytes transmitted per communication kernel. The lack of measurement tools constrains our understanding [24] of Scale-up network telemetry to the coarse application layer, rather than deeper network telemetry, such as packet-level latency or channel-wise throughput inside communication kernels.

To bridge this gap, we present FABRICPERF, a toolkit for *fine-grained, cross-layer* and *multi-faceted* Scale-up network measurement. FABRICPERF's design targets communication kernels, the networking code running on GPUs that generates Scale-up network traffic, transforming network telemetry into a GPU kernel profiling problem. FABRICPERF adopts a software-hardware co-design: it extends GPU kernel instrumentation tools [25] to enable transport-layer, packet-level telemetry and leverages memory profilers for physical-layer observability. FABRICPERF measures Scale-up networks without NIC/fabric/switch support with three key features: **Fine-Granularity:** FABRICPERF operates at the transport layer, the lowest software level available for developers. With GPU-Precision-Time-Protocol (GPTP) for clock synchronization, FABRICPERF measures packet-level latency and channel-wise throughput that are more fine-grained than application-layer profilers [48].

Cross-Layer: By modeling network communication as memory traffic, FABRICPERF also captures physics-layer insights (Tab. 1) by incorporating mature memory profilers [43] to characterize the physical implementation efficiency.

Multi-Faceted: FABRICPERF supports multiple network metrics (Tab. 1) by collecting packet-level timestamps, a unified metric

representation that can be reduced to latency ($T_{recv} - T_{send}$), or to throughput/packet rate (via bin counting).

FABRICPERF's hybrid design (§3) probes [25] the communication kernels to collect precise, segmented timestamps across packet lifecycles from being initiated by senders to entering fabrics to being received. With GPTP clock synchronization, these timestamps can be paired and aligned to derive precise packet-level metrics. Additionally, FABRICPERF models network flows as memory traffic, enabling memory profilers [43] to provide physical-layer insights.

We implement (§4) FABRICPERF on Linux for the NVLink ecosystem, consisting of three components: a GPTP clock synchronizer, a communication kernel prober, and memory profiling utilities. The prober based on neutrino [25] instruments communication kernels (Scale-up network code) to record and exchange timestamps (sent/received) for telemetry. The GPTP synchronizer runs before communication to align clocks across the whole Scale-up network domain. At the physical layer, the memory profiler characterizes the communication kernels' memory traffic and cache behavior.

We conduct comprehensive evaluations (§5) to validate the trustworthiness and overhead of FABRICPERF. Our results demonstrate that FABRICPERF achieves stability in clock synchronization (101.8ns jitter), reliability in measured latency and throughput (at ns-level), accuracy in memory modeling (only 6.86% error on average), and low overhead (~1% for offline mode and ~9% for online mode).

We systematically measure (§6) Scale-up network behaviors with FABRICPERF and discover two key findings: ① Scale-up network channels are highly imbalanced, with P99 latency 2x higher than P50, which degrades overall throughput due to longer end-to-end latency consumed by tailing channels. By employing work-stealing, we rebalance channels by 46% and improve *AllGather* throughput by up to 17.5GB/s. ② Underlying memory system and LLC exhibit poor performance (~0% hit rate) for Scale-up network buffers due to competing with other local buffer memory flows. By optimizing LLC eviction priorities to tune memory flows, we improve the LLC read hit rate by 34.86% and *ReduceScatter* throughput by 16.07GB/s.

Because Scale-up network stacks are still incomplete, FABRICPERF has limitations (§7) compared with well-established NIC-based measurement tools, such as an inability to directly measure network-layer details (e.g., routing). Nonetheless, as a fine-grained, cross-layer, and multi-faceted measurement toolkit for emerging, NIC-less Scale-up networks, we envision FABRICPERF as a useful tool and as a call to both research and industry communities to build more complete and efficient Scale-up network stacks. We have open-sourced FABRICPERF at github.com/open-neutrino/fabricperf to foster a global community for its continued development.

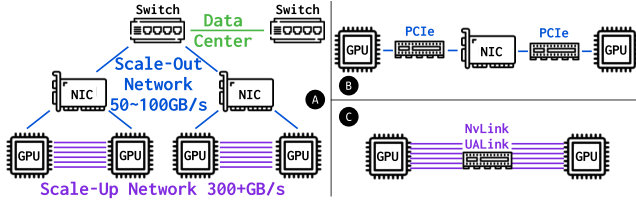


Figure 2: ML Network Paradigm. ① Hierarchical ML network consisting of Scale-up and Scale-out networks; ② Data path of Scale-out Network includes two PCIe passes; ③ Data path of Scale-up Network without PCIe involvement for high 300+GB/s bandwidth.

2 Background and Design Choice

The transition from GPU-NIC to GPU-Fabric architectures introduces distinct Scale-up network characteristics that the network community has yet to fully recognize. Thus, we present some key background in Scale-up network paradigm (§2.1), network stack (§2.2), underlying GPU memory system (§2.3), communication kernels (§2.4), and network measurements (§2.5) that lead to our design.

§2.1 Scale-up and Scale-out Network: As demonstrated in Fig. 2, the fundamental difference between Scale-up and Scale-out networks is the data path that leads to the divergence in bandwidths, semantics, and infrastructures. Scale-out networks (Fig. 2B), like *InfiniBand* [62] or *RoCE* [21], leverage the NICs connected with GPUs via local interconnects (PCIe). Therefore, native semantics of Scale-out networks are send and recv of packets, but the bandwidth (50-100GB/s) and latency ($\sim 100\mu s$) are bounded by the PCIe. Scale-out networks can be monitored using mature NIC network stacks and can adopt conventional protocols.

Scale-up networks simplify the data path to GPU→Fabric→GPU (Fig. 2C), accelerating bandwidth to TB/s and reducing latency to a few μs . These fabrics (*NvLink*, *InfiniteFabrics*, *UALink*) are directly attached to the GPU memory system. Thus, the *native* semantics of Scale-up networks are memory ld/st operations on addresses. Since NIC and OS are optimized out, Scale-up networks cannot be profiled via the NIC or the host network stack, and cannot apply established protocols.

Additionally, modern Scale-up networks incorporate switches (*NvSwitch*, *UASwitch*) to enhance its applicability in supporting arbitrary point-to-point communication, and scalability to larger network domains, e.g., *NVL72* [47]. Scale-up switches are commonly designed to be *non-blocking*, targeting the *fully-connected* topology and a unified memory space among different GPUs, but may also lead to pitfalls such as load balancing or congestion issues.

§2.2 Scale-up Network Stack: Since numerous infrastructures have been built to support the Scale-up network, we first classify these infrastructures into a four-layer network stack (Fig. 3) as inspired by the TCP/IP model.

Layers 1 and 2, i.e., the physics and the network layer, are proprietary black-box technologies that lack profiling support (exposing only byte counters [45] for monitoring). More importantly, there is no programmable interface, like SmartNIC/DPU or programmable switches, for custom development. Layer-1 and 2 expose their functionality via the memory semantics, i.e., ld/st 128/256bit data given unified addresses. Layer-3, the transport layer, is built on memory

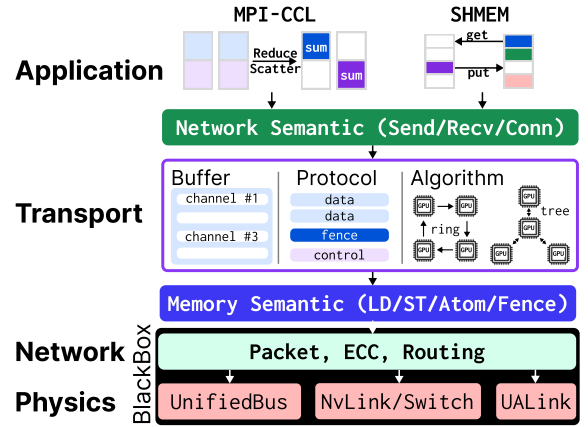


Figure 3: Scale-up Network Stack. Transport layer implements buffers and protocols to support network semantics (send/recv) upon memory semantics (ld/st) from black-box NET and PHY layers.

semantics to maintain network buffers, channels, protocols, and algorithms to provide reliable communication within the Scale-up domain. Layer-3 exposes its functionality as the network semantics, i.e., send/reduce/recv GBs of data via connections.

Layer-4, the application layer, is built above to support communication abstractions compatible with Scale-out networks for developers. A common abstraction is *collective communication* [50] that defines coordinated data exchange patterns across GPUs via Scale-up network, and has been widely applied in ML infrastructures, such as *Allreduce* for TP [59] and *AlltoAll* for EP [15]. Optimized collectives are pipelined (Layer-3) network semantics in ring/tree [51, 65] manner, following a common *recv→reduce→send* order. For instance, in *ReduceScatter*, each node *recv* chunks from the previous node *reduce* with local data, and send to the next node.

§2.3 GPU Memory System and Model: As the physical implementation of Scale-up networks, the memory system shapes the design of Scale-up networks. Most (NVIDIA/AMD) GPU memory systems have core-level L1 caches and device-level L2 cache(s) (LLC) before the DRAM (HBM/GDDR). As shown in Fig. 4A, cores and memories are connected by a *crossbar* (XBAR) that handles memory requests, including the internal (DRAM) and the external via fabrics (Scale-up network) or PCIe (CPU and NIC for Scale-out network).

In this architecture, the instruction and data path of inter-device st are combined to Core→LLC→Fabric→LLC(Peer), but ld needs a separate additional data path: LLC(Peer)→Fabric→LLC→Core (red dotted arrows in Fig. 4A). Though ld semantics can simplify point-to-point communications via inherent arrival support, st semantics are better suited for, and more widely used in, dependent collective communications of the *recv→reduce→send* manner. Therefore, the rest of the paper will focus on st-oriented communications. But our methodology can also be *simplified* to support ld-oriented semantics, as discussed in §7.

Most GPUs employ the *weak* memory model [46] where ld/st requests can be executed out-of-order to maximize throughput. To enforce strict orderings, a prerequisite for inter-device synchronization in networking, GPUs provide the fence instruction to flush the I/O buffers. For example, as shown in Fig. 4B, a fence applied to

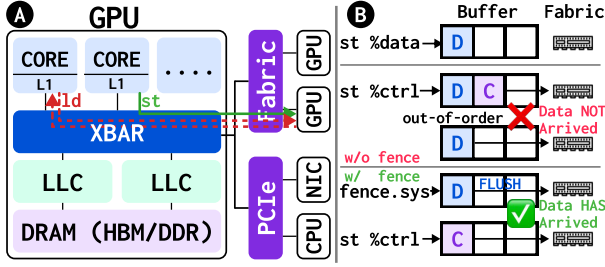


Figure 4: GPU Memory System (A) and fence (B). *st* only cross fabrics once but *ld* needs twice (instruction and data). fence postpone I/O operation to ensure correct ordering of data → *ctrl*.

the system-level store buffer will stall subsequent memory writes until the GPU store buffer is drained. This ensures all preceding writes are flushed into the fabric and globally visible; Consequently, when a remote core observes a later write, the arrival of all prior writes, e.g., data in Fig. 4B, is guaranteed.

§2.4 Kernels and Protocols: Communication kernels execute concurrently across GPUs and generate Scale-up network traffic. These kernels are executed within unified memory spaces across GPUs, which is achieved by exporting/importing page tables. Thereby, the memory system can distinguish outgoing memory requests and issue Scale-up traffic. Moreover, to be performant, communication kernels use parallel strategies and protocols that differ fundamentally from standard NIC-centric Scale-out networks.

This difference stems from the MTU offered by the underlying fabrics. By default, Ethernet/IP offers a 1500Byte MTU, so a small number of threads (typically <50) is sufficient to saturate the network. In contrast, as shown in Fig. 5A, Scale-up fabrics mostly provide only a 128Bit MTU (memory granularity) with much higher bandwidth, requiring many more threads to fully utilize the network. For example, NCCL [44] needs 10,240 threads to saturate the NvLink at 350GB/s on H100 for send/recv. These threads are organized in the GPU’s hierarchical parallelism; 64-1024 threads form a *block* that runs on the same core and coalesces memory writes in the same *local* memory pipeline. Thus, in Scale-up networks, a block naturally corresponds to a *channel* as the basic communication unit.

Small MTU also drives a different Layer-3 protocol design, i.e., how the *control plane* is incorporated to provide reliable communication for GB/TB-scale transfers over the MTU. For Ethernet/IP with 1500Byte MTU, most protocols use an *integrated*, 20Byte control plane at the beginning as a *header*. However, for Scale-up with only 8/16Byte MTU, injecting a header (even just a few bytes) into every message is prohibitively costly. Therefore, performant Scale-up protocols often use a *separate* control plane that is updated once after many 8/16Byte writes. To preserve integrity (i.e., control plane must arrive on or *after* the data plane), as shown in Fig. 5, protocols update the control plane at the end (a so-called *tailer*) of data transmission and after a fence, so that the control plane update is ordered after the preceding data writes.

§2.5 Network Measurement: Fundamentally, network measurements target latency, throughput, packet rate, and packet loss rate¹,

¹Scale-up fabrics are designed to provide reliable (lossless) delivery via error correction, so packet loss rate is often not a concern.

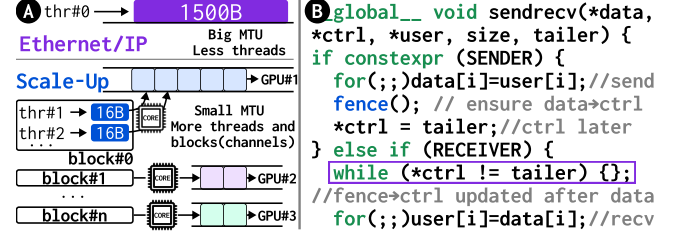


Figure 5: MTU/parallelism difference of ETH/IP and Scale-up (A) and example communication kernel (B). Scale-up network has smaller MTU and needs more (>10,000) threads; the fence’s ordering ensures when *ctrl* is matched, data has arrived.

which can be measured at different layers. And the key challenge is obtaining *fine-grained* measurements that explain performance.

Advanced, fine-grained measurement frameworks often break the above metrics into smaller *segments* that correspond to specific components or procedures along the data path. For example, Swift [30] decomposes *RTT* into 7 segments spanning host ↔ NIC and NIC ↔ switch interactions. Such segmented measurements have become a foundation for informing modern protocol designs [19, 32, 35], because it can efficiently diagnose performance degradations into specific domains, e.g., the host, the NIC, and the switch.

Notably, as the physical implementation, hardware also plays a crucial role in network measurements. For instance, *hostCC*, *shRing*, and *CEIO* [2, 34, 52] shows that memory contention and LLC eviction of network data could harm high-bandwidth (10+GB/s) network performance. Scale-up networks with the highest bandwidth (TB/s/GPU) may have a similar or even more severe impact on the memory system, which shall also be considered.

Overall, fine-grained network measurement is a hardware-software co-design problem, even in the network domain: precise timestamping depends on hardware support. For example, Swift utilizes NIC PTP clock to record packet *entering/leaving* and switch timers to record *resident time*. In Scale-up networks, such hardware support may be unavailable or exposed through different interfaces. Thus, a major challenge of FABRICPERF is how to achieve precise measurements using available interfaces, i.e., memory semantics.

3 FABRICPERF Design

FABRICPERF’s objective is to formulate a fine-grained measurement framework, with packet-level precision, and coverage comparable to the host- and NIC-based network stacks, yet without additional hardware (e.g., NIC) support.

The key design question is: at *which* layer should FABRICPERF operate and *how* should it obtain observability? Among all layers shown in Fig. 3, we choose the *transport layer* as the *primary* profiling interface because the application layer is not packet-granular and PHY/NET layers are black boxes, while the transport layer implements key network mechanisms, e.g., packetization and protocol control logic, and is the lowest software layer that can be inspected without additional fabric or switch support. Moreover, to better understand physical behaviors, we additionally model network communication as memory traffic, and employ *memory traffic* as the *secondary* profiling interface. This choice is motivated by the

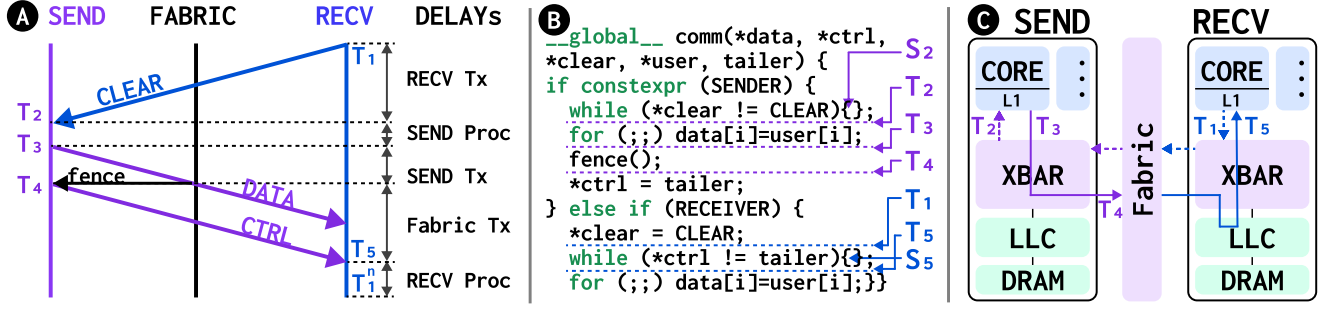


Figure 6: FABRICPERF Packet-Level Timestamping in network model (A), communication kernel code (B) and hardware (C).

fact that Scale-up fabrics communicate via memory semantics and pipelines, and that memory profilers (e.g., cache and addresses) are more mature and informative than fabric-specific ones that often only report byte counts.

The *second* design question is: *what* information should FABRICPERF collect to cover the wide range of metrics (§2.5)? We identify that *fine-grained packet-level timestamps* can serve as a unified metric representation that derives many metrics. For example, given the packet sending time (T_{send}) and arrival time (T_{recv}), the packet-level latency is $T_{recv} - T_{send}$. Similarly, throughput and packet rate can be calculated by aggregating arrival timestamps (bin counting) by T_{recv} , w.r.t. the packet size and count, respectively.

Thus, as shown in Fig. 1, FABRICPERF is centralized with a software (protocol) and hardware (memory) co-design that uses protocol-level probing to measure packet-level timestamps, and models network communication as memory traffic to understand physical-layer behaviors. Under this design choice, we present three key designs towards precise (§3.1) and synchronized (§3.2) packet-level timestamping, and comprehensive physical memory analysis (§3.3).

3.1 Precise Packet-Level Timestamping

We now propose a collection method that is both precise and implementable for packet-level timestamps of Scale-up networks at the transport layer. From the communication kernel example (Fig. 5), we identify many network procedures (e.g., send/recv) whose code execution timestamps, particularly the protocol logic spanning the control plane and the data plane, can serve as packet-level timestamps. For example, duration(s) of the while loop on the control plane (the purple box in Fig. 5B) can represent a *proxy* for packet-level latency since this loop corresponds to the process by which the control plane becomes matched, i.e., packets received.

Therefore, our *first* design for *precise* timestamping is to probe communication kernels, analogous to eBPF [14] for the host (OS) networking stack. In Scale-up networks, the “network stack” is the transport layer and communication kernels running on GPUs. FABRICPERF attaches *small* timing probes to *exact* locations inside communication kernels and executes them during network operation, so that packet-level timestamps can be measured at runtime.

However, we identify that the above example of T_{recv} is not sufficiently precise because it does not capture the exact sending time (T_{send}). Therefore, inspired by segmented measurement tools, e.g., Swift [30], and simplified from the group communication model (Fig. 16), our final design for precise timestamps is to decompose

latency (or *RTT*) into finer-grained segments as follows (Fig. 6):

T_1 Recv Tx Timestamp: Receiver issues CLEAR (could be updating steps in practice), indicating that its receiver buffer is ready.

T_2 Send Rx Timestamp: Sender observes (and matches) CLEAR and starts sending data to receiver’s buffer.

T_3 Send Tx Timestamp: Packet leaves the sender core.

T_4 Fabric Tx Timestamp: Packet enters the fabric at the sender side (after fence), then the control plane is updated.

T_5 Recv Rx Timestamp: Receiver observes the packet arrival, i.e., the control plane becomes matched and starts processing data.

S_2 and S_5 : Spinning counts while waiting for T_2 and T_5 .

Compared to de-facto RTT segmentations [30], we make three changes: ❶ We move the receiver’s ACK to the beginning as CLEAR. Without an NIC buffer to cache incoming packets, the sender can only transmit after the receiver indicates an available buffer (similar to $cwnd = 1$). ❷ We add S_2 and S_5 to check if the actual arrival time precedes when the polling thread returns to check (T_2 , T_5). If $S_2 = 1$ or $S_5 = 1$, the actual arrival time of CLEAR/packet may precede T_2 , T_5 . Without hardware-level arrival timestamps, we can only exclude these cases from latency measurements but we still include them for throughput and packet rate calculation. ❸ We exclude CLEAR enter fabric timestamps, because (i) CLEAR has no race condition as DATA and CTRL, so fence is not needed in practice and we cannot timestamp; (ii) CLEAR message is usually tiny as 64bit (pointer updates) whose transmit time ($<1\mu s$) is smaller than fence overhead ($\sim 1.5\mu s$), making it hard to be reliably timed.

3.2 Synchronizing Timestamps across GPUs

A remaining challenge of §3.1 is that packet-level timestamping in the network spans two devices (T_1 and T_5 on the receiver, but T_2 , T_3 , and T_4 on the sender), while probes execute only locally on each device. By default, timestamps (GPU clocks) are not synchronized between GPUs. Thus, metrics derived solely from timestamps (e.g., latency as $T_5 - T_4$) are unreliable without *precise* clock synchronization across the Scale-up network domain.

However, synchronizing GPU clocks across devices is challenging and mature solutions (e.g., ptp41 [17]) cannot be engineeringly ported. This is because GPUs lack OS support to directly access (open) the NIC’s PTP clock and even if such access were made possible, accessing the NIC clock over PCIe ($\sim 1\mu s$) would impose too much overhead for Scale-up networks with sub-10- μs latency.

Therefore, our *second* design is a GPU-native Precision Time Protocol (GTP) that uses GPU device clocks and the Scale-up memory

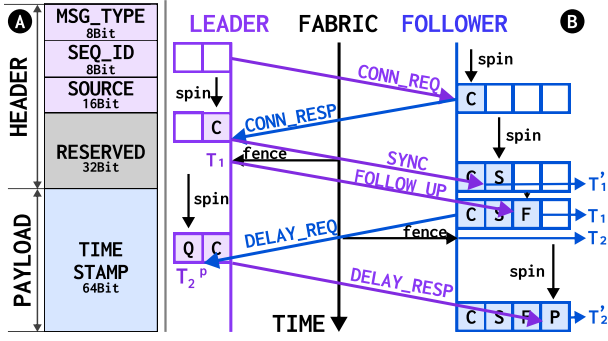


Figure 7: GPTP Message Design (A) and Protocol (B). The message is packed to 128bit for *atomic* transmission over fabrics. We add *CONN* before *SYNC* message for accurate T_1 measurement.

semantics. We design the synchronization mechanism between the boundary (leader) clock and ordinary (follower) clocks using *Sync*, *Follow_Up*, *Delay_Req* and *Delay_Resp* messages. Unlike PTP, which uses TCP/IP messages [4], we specially design GPTP messages to be only 128bit (64bit header and 64bit timestamp payload), as shown in Fig. 7A, so they can be transmitted *atomically* (since MTU is 128bit) by fabrics (§2.3). Another distinction of GPTP messages is that we omit the explicit *destination* field: Under scale-up memory semantics, that destination is implicitly determined by the address offset in remote writing (store).

To fit memory semantics, we divide messages into two separate buffers: [*Conn_Req*, *Sync*, *Follow_Up*, *Delay_Resp*] in the followers' buffers and [*Conn_Resp*, *Delay_Req*] in the leader's buffer. Importantly, this separation ensures that the receiver-side spinning (polling) is limited to local memory rather than over the fabric, thereby reducing overhead and limiting errors caused by latency variations. Since fabrics cannot provide hardware-level arrival timestamps like NICs, we add a handshake (*Conn_Req*, *Conn_Resp*) before *Sync* to ensure followers are actively polling for the *Sync* message so that we can measure T'_1 accurately. Lastly, we use a special device-level fence (with less scope and overhead than the system-level fence in Fig. 6) to *approximate* precise message-send times for T_1 and T_2 measurements.

3.3 Network as Physical Memory Traffic

An additional perspective on network measurement is the behavior of the underlying hardware. However, the lack of NICs in the Scale-up network makes traditional network-hardware profiling approaches, e.g., packet queues, inapplicable. Therefore, we propose a novel approach of modeling the network communication as memory traffic to leverage mature hardware-based memory profilers. This approach is motivated by the insight that fabric hardware exposes communication through memory semantics and utilizes memory pipelines as (part of) physical implementation (§2.3).

However, engineeringly using existing memory profilers [45] is impracticable because communication kernels introduce additional memory traffic beyond the fabric itself, and hardware profilers cannot distinguish memory flows. Accordingly, our *third* design is a comprehensive model for estimating the memory traffic from network traffic from two perspectives as follows:

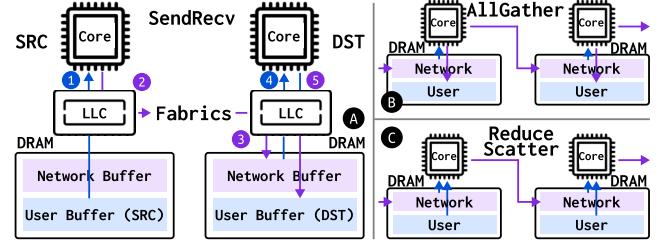


Figure 8: Communication Kernel Memory Flow includes network and user buffers, with read (blue) and write (purple) flows for collectives: A SendRecv; B AllGather; C ReduceScatter.

Table 2: Theoretical Memory Traffic and LLC Behavior Model for *SendRecv* (SR), *AllGather* (AG) and *ReduceScatter* (RS).

	SR	SR(DMA)	AG	RS
Read memBw	2x	1x	$(1+1/N)x$	2x
Write memBw	3x	2x	3x	$(2+1/N)x$
L1 Behavior	bypass	bypass	bypass	bypass
LLC Read Hit %	1/2	0%	100%	1/2
LLC Write Hit %	2/3	1/2	2/3	100%

Ideal Memory Traffic: We model the ideal memory traffic by taking the fabric bandwidth (fabBw, in GB/s as measured in §3.1) as the input to estimate the corresponding memory traffic (memBw, to be measured by memory profilers) as the output. For sending, as shown in Fig. 8A, we observe the following memory traffic:

- 1 (*SendRecv*, *ReduceScatter*) The sender reads from the local source buffer, creating x fabBw of *read* traffic.
- 2 The sender writes to the receiver buffer through the fabric, creating x fabBw *write* traffic.

For receiving, we identify the following memory traffic:

- 3 Remote writes (from the sender) arrive into the receiver's buffer via the fabric, creating x fabBw of *write* traffic;
- 4 Receiver cores read the network buffer to check/validate, creating x fabBw of *read* traffic;
- 5 (*SendRecv*, *AllGather*) Receiver cores write data to the local destination buffer, creating x fabBw of *write* traffic.

4 and 5 *disappear* when using the DMA-style buffers [41], i.e., writing directly into the user buffer (similar to kernel-bypass I/O [16]). However, 4 and 5 also handle essential logic, e.g., reduction, so DMA-style buffers only apply to *SendRecv* and *AlltoAll*. Moreover, different rank-based communication kernels follow different logic (Fig. 8), and have *slightly* varied traffic patterns w.r.t. the number of ranks (N): *AllGather* performs an additional $1/N$ fraction of reads per GPU for the source (1), and *ReduceScatter* has an extra $1/N$ writes per GPU to store results (5), as summarized in Tab. 2.

Cache Behavior: We include cache effects in our model because caches have much lower latency than DRAM. Therefore, network latency can be significantly reduced if network data is written into and consumed from caches [2, 34, 52]. Notably, network data is external to the core, so the *per-core* L1 caches have essentially 0% hit rate and should be bypassed [12]. Thus, we build a model to estimate the L2 (LLC) hit rate from the memory traffic via classifying memory traffic by locality: Network buffers are reused across packets (high locality) so they shall be persistent in cache (100%

hit), whereas user buffers are *typically* accessed in a streaming manner (reading/writing only once, so 0% hit). Accordingly, we have the following observations for the memory traffic above: ❶ Source reading is streaming with no locality (0% hit). ❷ Writes to remote memory are expected to be cached since there is no corresponding local GMEM to be replaced. ❸ Remote writes are streaming (0% hit) when targeting user buffers, but shall be persistent (100% hit) when targeting network buffers. ❹ Network buffer reads are persistent (100% hit). ❺ Destination writes are streaming (0% hit). We, thus, estimate the theoretical LLC behavior for each collective in Tab. 2.

4 FABRICPERF Implementation

We implement FABRICPERF on Linux for the CUDA ecosystem with NVLink as the Scale-up network. Our implementation, according to our design, consists of three components: ❶ Communication kernel prober based on Neutrino [25] to collect packet-level timestamps; ❷ GPTP to synchronize packet-level timestamps collected in different devices; ❸ Memory traffic profiler based on CUPTI [43] to measure the memory traffic from the network traffic. As shown in Fig. 9, we wrap these modules as a Neutrino plugin used in CLI:

```
$ fabricperf -mode=latency mpirun nccl/sendrecv ...
```

4.1 Probing Communication Kernels

Packet-level timestamping requires probing GPU communication kernels at the exact line of code, *e.g.*, after the while loop for T_{recv} (Fig. 6). However, probing GPU kernels is challenging since there are no in-place solutions, *e.g.*, eBPF [14] for the OS, and the communication involves cooperative probes on multiple GPUs, blocks, and threads. More importantly, since our target (timestamps) is within the time domain, collection overhead (additional time) matters for accuracy. For example, using the built-in `printf` would slow down the kernel by 5x and greatly affect the accuracy of metrics.

In FABRICPERF, we leverage a recent GPU kernel probing tool, Neutrino [25], which attaches small snippets (probes) to GPU programs at the assembly layer, offering instruction-level granularity high programmability, low overhead, and versatility across the time domain (allowing reading device clocks). We use Neutrino to match and attach timing probes to particular instructions, such as the fence to record T_4 , the bra (assembly form of while) to record T_2 and T_5 , and the first st to record T_1 and T_3 .

We also extend a plugin system for Neutrino to allow adding host-side hooks and support: ❶ MPI integration for cross-rank GPU identification and trace collections; ❷ Exchange buffers, in addition to Neutrino maps, via unified memory mappings for GPTP and $T_1 - T_5$ timestamp exchanges; ❸ Non-blocking kernel execution with replays to enhance stability. However, Neutrino's eBPF-like map abstraction in DRAM is still too slow for FABRICPERF. Therefore, we support two modes for different use cases: (i) online feedback (Fig. 9B) and (ii) offline collection (Fig. 9A).

Online Feedback: Network protocols' runtime reconfiguration requires measurement metrics to be instantly available within communication kernels. FABRICPERF supports the online mode for such needs by exchanging packet-level timestamps and performing necessary calculations, *e.g.*, latency, on-the-fly. As shown in Fig. 9B, we implement this by reusing the 128bit GPTP message format (Fig. 7A) and taking $T_1 - T_5$ as the payload. For the availability of T_1 and

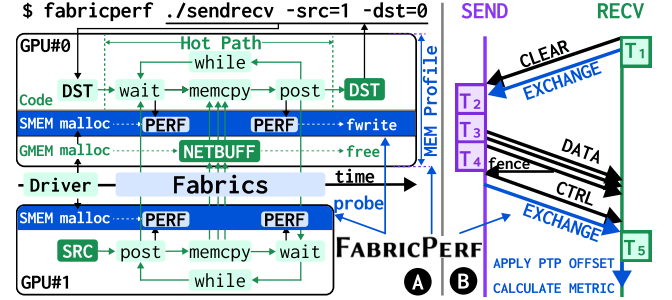


Figure 9: FABRICPERF Implementation instruments the communication kernel to collect packet-level timestamps and uses memory profilers for physical insights. Supports ❶ **Offline mode** using SMEM to reduce overhead. ❷ **Online mode** exchanging timestamps to calculate metrics instantly.

T_5 , we send the timestamps right after the CTRL or CLEAR message, for which the receiver will apply the GPTP clock offset, perform the calculation, and drop timestamps.

Offline Collection: There are other profiling needs that require no runtime availability but instead the complete trace (*e.g.*, all $T_1 - T_5$) for posterior analysis, which FABRICPERF supports as the offline mode. To avoid perturbing the networking hot path, we exploit the fact that shared memory (SMEM, the programmable part of L1 cache) is not utilized in networking (Tab. 2), and use SMEM as the temporary buffer (Fig. 9A). Thus, costly saving to GMEM only happens at the epilogue of communication kernels and reduces per-event timestamping overhead in the hot path from 200+ns (GMEM) to ~20ns (SMEM), improving efficiency and measurement fidelity.

4.2 GPU-Precision-Time-Protocol

We implement GPTP with reference to the IEEE-1588 standard [4]. Due to GPU/fabric limitations, we make several key modifications:

❶ Since accessing the NIC's PTP clock is impossible or inefficient, we use the GPU built-in device clock (`%globaltimer` for NVIDIA and `S_MEMREALTIME` for AMD GPUs) whose frequency cannot be adjusted (no `adjtime`). ❷ Since we can only correct clock offsets, we assume homogeneous GPUs and clocks. Accordingly, we simplify the master election algorithm (BMCA [38]) and select any GPU (typically the first GPU-0) as the leader. ❸ Since fabrics cannot annotate the message passing time, our implementation focuses on the boundary/ordinary clock and does not include the transparent clock in PTPv2 [4]. Finally, we implement GPTP in assembly for integration with our probing tool (§4.1), as well as in a header-only library for standalone use.

4.3 Memory Profiler and Utilities

The last part of our implementation is the hardware-based memory profiler that measures the memory pipeline serving the Scale-up network traffic. We mainly incorporate CUPTI [43] to capture XBAR and DRAM read/write bandwidth, LLC hit rate, fabric send/recv byte counters (NVLink), etc.

To sort out the impact of different cache line sizes, we use the XBAR bandwidth (the total input of LLC) and the DRAM bandwidth (the output of LLC) to measure the LLC hit/miss rate at the byte

level, rather than at the cache line level. Specifically, on NVIDIA GPUs, XBAR write bandwidth is measured as the (missed) output of the L1 cache and does not include the external write traffic from the fabric, *i.e.*, ② in Fig. 8. Therefore, we manually add the application-level fabric receiving bandwidth as a compensation term.

5 FABRICPERF Evaluation

As an initial measurement tool for Scale-up networks, our evaluation primarily aims to verify FABRICPERF’s correctness, metric accuracy, and overhead, and demonstrate how to use it to better understand scale-up networks in practice.

Our evaluations use two testbeds: ① H100 NVL8 (single-node) featuring 3.5TB/s memBw, fully connected via four NvSwitch chips with a theoretical, bi-directional 900GB/s netBw. ② GB200 NVL16 (multi-node, part of NVL72 with four NVL4 nodes) featuring 16TB/s memBw, fully-connected via 9 NvSwitch trays (in total 18 NvSwitch chips) with a theoretical, bi-directional 1.8TB/s netBw. Both testbeds share the fully-connected topology with the same *one-layer switch layout*, and use 31.25 MHz (32ns granularity) hardware clocks.

We use NCCL (v2.28.9) [44], the de-facto Scale-up framework, and its test-suite `nccl-tests` (v2.17.0) [48]. Among all collectives, we pick three representatives, *SendRecv*, *AllGather* and *ReduceScatter* because *SendRecv* can be extended for *AlltoAll*, while *AllGather* and *ReduceScatter* can be composed to *AllReduce*. To generate diverse workloads, we mainly tune NCCL_CTAS (an environment variable) to adjust the number of channels, thereby indirectly controlling bandwidth. All collectives are run in *Ring* [51] algorithm.

5.1 GPTP Stability and Correctness

We evaluate GPTP synchronization along two dimensions: Stability across time and relative correctness/consistency across the domain. **Stability:** We verify stability by continuously running GPTP (attached to communication kernels to simulate real patterns) and tracking the variance of clock offsets (*Jitter*) over time. We treat each GPTP independently, since we cannot use `adjtime` [33] to converge clocks. Results in Tab. 3 show that GPTP effectively aligns clock *Offsets* along different ranks and different nodes, often on the order of *ms*. Yet offsets vary over time (kernels), yielding an inter-kernel *Jitter* J_{inter} at 35.96 μ s on average. This is far beyond normal PTP *Jitter* at sub- μ s-level [40]. By comparing subtle frequency differences (*Drift*), we attribute the high *Jitter* to the misaligned clock frequency over different nodes ($D_{inter} = 1.541\mu$ s/run on average), while intra-node clocks are highly aligned ($D_{intra} = 0.107$ ns/run).

To provide a reliable bound, we also measure the intra-kernel stability and find that the intra-kernel *Jitter* (J_{intra}) becomes stable at 101.8ns on average. which is far below typical sub-10- μ s packet latencies. Overall, Tab. 3 suggests that PTP synchronization for each kernel is necessary for accuracy. Importantly, GPTP only takes 5-10 μ s, which is orders of magnitude faster than CPU PTP (seconds), and incurs only 1-2% overhead on the Scale-up communication.

Relative Correctness: We evaluate the relative correctness (all clock pairs synchronized) by validating that offsets obtained via direct measurements should match offsets inferred through indirect paths (*e.g.*, via the master). Because communication patterns can be arbitrary, while PTP synchronization is efficiently implemented in

Table 3: GPTP Stability Evaluation on GB200 NVL16. GPU 1,4,8,12 represent different nodes (each of 4 GPUs) w.r.t. GPU 0.

GPU	Offset	J_{intra}	J_{inter}	Drift	Lag
Avg	N/A	101.8ns	35.596 μ s	N/A	724.3ns
0→1	-545.4ns	65.8ns	72.3ns	-0.055ns/run	729.7ns
0→4	7789.069ms	66.4ns	87.968 μ s	3047.414ns/run	710.2ns
0→8	741.666ms	128.7ns	10.067 μ s	348.733ns/run	706.7ns
0→12	-302.268ms	124.2ns	7.040 μ s	243.882ns/run	715.6ns

Table 4: GPTP Synchronization Correctness. GPTP can correctly sync all clocks within the domain with max $\epsilon = 364.6$ ns.

Offset $_{i \rightarrow k}$	Offset $_{i \rightarrow j}$	Offset $_{j \rightarrow k}$	Error (ϵ)
1→10 (741.660 ms)	1→4 (7789.069 ms)	4→10 (-7047.410 ms)	364.6 ns
0→10 (741.659 ms)	0→4 (7789.069 ms)	4→10 (-7047.410 ms)	359.8 ns
1→6 (7789.070 ms)	1→4 (7789.069 ms)	4→6 (73.4 ns)	357.1 ns
Results Truncated			
9→5 (7047.402 ms)	9→14 (-1043.935 ms)	14→5 (8091.336 ms)	0.04 ns

Table 5: Latency/Throughput accuracy evaluation. Numbers are organized in H100 (1GB data) / GB200 (2GB data) manner.

Latency	<i>SendRecv</i>	<i>AllGather</i>	<i>ReduceScatter</i>
Latency	6.862/6.381 μ s	4.397/13.564 μ s	13.207/11.414 μ s
Ref [24]	~ 6 μ s	~ 6 μ s	~ 6 μ s
Throughput	366.37/698.18GB/s	335.31/684.77GB/s	320.63/679.51GB/s
Ref [48]	360.67/690.20GB/s	319.24/680.22GB/s	317.14/680.22GB/s

Table 6: Segmented Latency Component Micro-benchmarks.

Device	<i>Recv-TX</i>	<i>Send-Proc</i>	<i>Send-TX</i>	<i>Fabr-TX</i>	<i>Recv-Proc</i>
H100	595.9ns	26.88 μ s	1.68 μ s	677.5ns	28.63 μ s
GB200 $_{intra}$	1.29 μ s	22.30 μ s	3.14 μ s	1.08 μ s	33.81 μ s
GB200 $_{inter}$	1.15 μ s	22.29 μ s	3.11 μ s	1.13 μ s	33.73 μ s

a *master-slave* fashion, we evaluate the offset errors using triangulation: for any GPU i, j and k in a synchronized GPTP domain, we compute $\epsilon = \text{Offset}_{i \rightarrow k} - (\text{Offset}_{i \rightarrow j} + \text{Offset}_{j \rightarrow k})$. Results in Tab. 4 highlight that our GPTP correctly synchronizes all clocks within the domain, with an average triangulation error of $\epsilon = 106.45$ ns.

5.2 Metrics Accuracy

Network Metric: We validate the correctness of FABRICPERF’s network metrics by comparing our aggregated metrics against independent sources, such as averaged latency against reference values reported by NCCL [24] and throughput against application-layer measurements [48]. Tab. 5 shows that our throughput closely aligns with application-layer results and is *slightly* higher because FABRICPERF measures within the device and excludes host-side overhead. Our measured latencies are only *partly* (only *SendRecv*) aligned with NCCL reference values [24] because we find that latency varies with collective algorithm, where *ReduceScatter* are significantly higher due to reduction, and network configuration, while [24] does not report the corresponding setup. Thus, we can only broadly measure latencies under varied configurations (§6.1) and report the average here as a comprehensive evaluation.

Segmented Micro-benchmark: To evaluate the effectiveness of our segmented latency design (§3.1 and Fig. 6), we conducted micro-benchmarking based on GPTP (a standard echo setup) by adding a

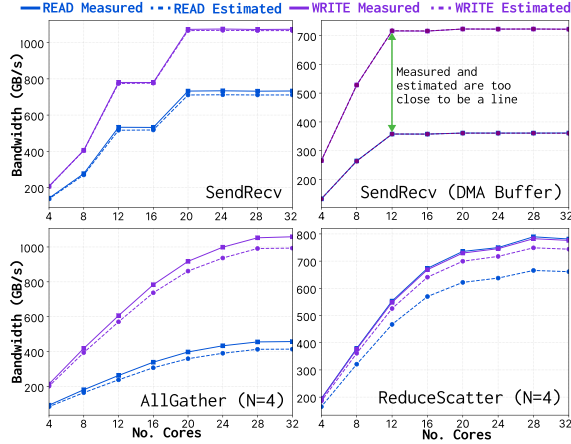


Figure 10: Memory Model Accuracy. Estimated (dotted) and measured (solid line) flows are aligned (mean error 6.86%).

4MB payload (the typical amount of Scale-up communication kernels per core and per turn). Our results in Tab. 6 show *Recv-TX* is small (0.6–1.3 μ s) for tiny 64bit message, and *Send-TX* demonstrates a larger fence cost (1.6–3.1 μ s). *Send-Proc/Recv-Proc* mutually match for memcpy for 4MB, while *Recv-Proc* takes longer than *Send-Proc* due to the additional memory-consistency need for remote writing. GB200 NVL16 has bigger *Recv-TX*, *Send-TX* and *Fabr-TX* than H100 NVL8, demonstrating longer data paths in larger domains. GB200_{intra} results closely match with GB200_{inter}, highlighting the consistency within *fully-connected* topology.

Memory Traffic Model: We verify the accuracy of our memory-traffic model (Tab. 2) by validating the consistency between measured memBw from hardware profilers (solid lines in Fig. 10) and estimated memBw derived from the fabric bandwidth (fabBw, dotted lines in Fig. 10). Our results in Fig. 10 demonstrate strong consistency between the estimated and measured memBw across both reading and writing flows, communication kernels, configurations (e.g., number of cores), and fabric bandwidths, with the biggest difference observed for ReduceScatter’s reading flow at $\sim 15\%$ and an average error 3.17% for the rest. This consistency supports the correctness of our memory-traffic model and motivates using it to interpret richer memory metrics (e.g., cache behaviors in §6.3).

5.3 Measurement Overhead

We evaluate the measurement overhead of FABRICPERF to communication kernels, with a particular focus on the network hot paths (the core logic in Fig. 6B) where added probes can perturb packet timing and thus impact timestamp precision. Our evaluation runs the same communication kernel under three configurations: (i) no measurement (*Ref.*), (ii) *Online* latency measurement, and (iii) *Offline* timestamp collection. For each run, we attribute overheads to three parts, *Prologue* (for GPTP synchronization and preparation, *not sensitive to overhead*), *Network Hot Path* (packet sending and receiving, *sensitive to probing overhead*), and *Epilogue* (for saving telemetry, *not sensitive to overhead*). Results in Tab. 7 show high efficiency of FABRICPERF: Offline collection only introduces 0.6% ($\sim 20\mu$ s) overhead on the network hot paths while enabling precise timestamping. Online measurements offer instant latency

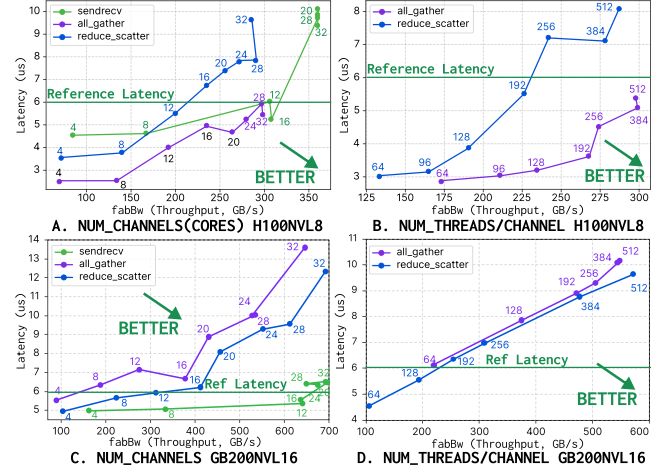


Figure 11: Scale-up Network Dynamics. Latency (y-axis) and Throughput (x-axis) under Varying Configuration: No.Channels (A, C), Threads per Channel (B, D) on H100 NVL8 and GB200 NVL16.

Table 7: FABRICPERF Measurement Overhead. Offline mode has lower ($\sim 1\%$) overhead than Online mode ($\sim 9\%$) w.r.t. Reference.

Mode	Prologue	Hot Path	Epilogue	Cost _{send}	Cost _{recv}
Ref.	N/A	$\sim 2.87ms$ (100%)	N/A	N/A	N/A
Offline	13 μ s	$\sim 2.89ms$ (+0.6%)	1.13 μ s	5 μ s	13 μ s
Online	13 μ s	$\sim 3.13ms$ (+8.9%)	1.03 μ s	325 μ s	138 μ s

feedback at 8.9% ($\sim 463\mu$ s) overhead, which can be further reduced with packet sampling [22] than the *all* packet probing used here.

6 Analyzing Scale-up Networks via FABRICPERF

In this section, we broadly measure Scale-up networks via FABRICPERF to characterize their runtime behavior. We first study how latency and throughput vary with network configurations (§6.1), followed by two case studies on channel imbalance (§6.2) on H100 and cache inefficiency (§6.3) on GB200.

6.1 Network under Varying Configurations

We characterize Scale-up network dynamics using two metrics, *i.e.*, fabric latency ($T_5 - T_4$) and throughput (sum from each channel). We focus on how these dynamics vary with different configurations, challenging the previously common assumption that Scale-up networks exhibit a nearly fixed $\sim 6\mu$ s latency [24]. We study four major factors: different network, collectives, number of channels and number of threads per channel that expands parallelism (§2.4). **Different Collectives:** We find that different collectives exhibit different latency behaviors under the same configuration, though they present similar throughput. This difference may be related to the different packet processing memory access patterns. For instance, *SendRecv* (DMA buffer) removes local copies and can have higher throughput with fewer channels than *AllGather/ReduceScatter*.

Channels: By varying the number of channels (in Fig. 11A/C), we observe an approximate Pareto Frontier trade-off between latency and throughput: When < 8 channels on H100, increased throughput comes with little latency penalty, suggesting underutilized fabrics.

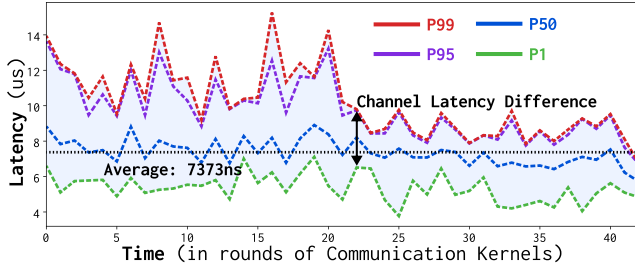


Figure 12: Latency Distribution across Channels (y-axis) and Time (x-axis). Channels are imbalanced with high P99.

Beyond 8 channels, the network becomes saturated, and additional concurrency increases latency. After 20 channels, adding channels sacrifices latency severely with only marginal throughput gains.

Threads: Varying threads per channel shows a similar throughput-latency trade-off that starting from 128 threads/channel throughput gains substantially sacrifice latencies. We also find a special 384 threads setup that improves throughput with a small latency increase, revealing understudied thread parallelism (§2.4) impact to Scale-up networks.

Network: Different Scale-up networks (GB200 *versus* H100) have varied bandwidth that requires diverse parallelism. For instance, GB200 needs 32 channels to saturate the network instead of 24 channels on H100. Moreover, average latency on GB200 (5-13 μ s) is higher than H100 (2-10 μ s), demonstrating longer data paths and more complicated switch internals needed for higher bandwidth.

6.2 Channel Imbalance

We now break down averaged latencies (points in Fig. 11) for more in-depth analysis. As discussed in §2.4 and Fig. 5, Scale-up communication is parallelized across many channels, and large workloads need to be partitioned for sequential transmission as packets and kernels over multiple rounds. This model expands two dimensions of characteristics: ❶ How do network dynamics (latency) vary across time (rounds of communication kernels)? ❷ How is latency distributed across different channels (cores)?

As shown in Fig. 12 (x-axis), the network latency is skewed in the time domain. We observe a relatively higher latency, up to 14 μ s for the first 20 rounds. Then latency decreases, particularly in the P99 latencies (upper bound experienced by the application layer) dropping from $\geq 10\mu$ s to sub-10 μ s, indicating a network warm-up phase before reaching peak performance.

Fig. 12 (y-axis) further demonstrates substantial channel-level imbalance. Some channels experience $\sim 2\times$ higher P99 latency (up to 14 μ s) than typical channels (P50/median around 7 μ s, and P1 as low as 4 μ s). Moreover, to confirm that the channel imbalance is not due to the unavoidable saturation under full network utilization, we also study latency versus the number of channels and the results in Fig. 17 show that channel imbalance arises very early at 12 cores with ~ 200 GB/s, far from saturation (300+GB/s on H100).

Note that channels here correspond to cores/memory pipelines, rather than PHY link/port(s), with the mapping instructed by the network layer. Thus, observed imbalance suggests *feasible* inefficiencies in current network layer designs, e.g., port selection, scheduling, which, regrettably, lacks network-layer details to confirm.

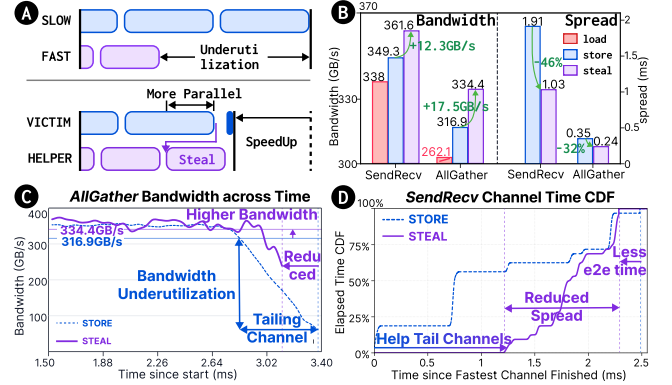


Figure 13: Exposed Channel Imbalance and Work-stealing on H100: ❶ Demonstration of varied elapsed time and work-stealing impact; ❷ Work-stealing improve busBw by 17.5GB/s (detailed in ❸) and reduce channel spreads by 46% (detailed in ❹).

Algorithm 1: Work-Stealing Channel Balancer.

Input: $S_c = (\text{progress} = 0, \text{end} = N, \text{stolen} = F, \text{hDone} = F)$
Param: max helpers H , steal fraction Q , num chunks N
Channel c Parallel for $k \leftarrow 0$ to $N - 1$ do
 if $k < S_c.\text{end}$ then
 recvSend(chunk k); $S_c.\text{prog} \leftarrow k + 1$;
 else if $S_c.\text{stolen}$ and $k = N - \lceil (N - S_c.\text{prog}) * Q \rceil$ then
 wait($S_c.\text{helperDone} = T$); $S_c.\text{prog} \leftarrow N - 1$; break;
 end
if atomicAdd(h , 1) $< H$ and not $S_c.\text{stolen}$ then
 victim channel $v \leftarrow \min(S_c.\text{prog})$
 if atomicCAS($S_v.\text{stolen}$, F , T) succeeds then
 $S_v.\text{end} \leftarrow N - \lceil (N - S_v.\text{prog}) * Q \rceil$
 recvSend(chunks [$N - \lceil (N - S_v.\text{prog}) * Q \rceil$, N))
 $S_v.\text{helperDone} \leftarrow T$; // hDone := helperDone
 end
end

Though optimized communicators [44] use parallel workgroups within channels to try to hide the channel imbalance, we still observe a tailing effect of channel completion time (blue dashed line of Fig. 13D) due to channel imbalance. Consequently, fabric bandwidth ends from a straight to a descending line (Fig. 13C), resulting in network underutilization and degraded overall throughput.

Though Scale-up fabrics lack network layer support for load balance opportunities, we propose a work-stealing-based channel balancer that acts in the protocol layer via customizing NCCL device kernels [48]. Our channel balancer introduces a global progress monitor to allow faster channels to check and steal slower channels' remaining workload, as shown in Fig. 13A and Algorithm 1. Results in Fig. 13B highlight the effectiveness of our channel balancer in reducing channel spreads by 46% (from blue line to purple line in Fig. 13D) and improves end-to-end *SendRecv/AllGather* throughput (purple line of Fig. 13C) by 12.3GB/s and 17.5GB/s beyond optimized communicators [48] (red and blue boxes in Fig. 13B), highlighting the practicality of FABRICPERF's insights in optimizing real Scale-up networking workload.

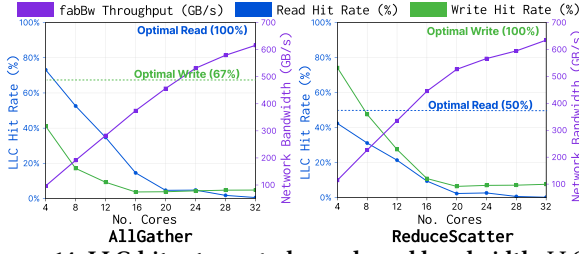


Figure 14: LLC hit rate w.r.t. channels and bandwidth. LLC hit rates quickly drop to the worst ($\sim 0\%$) with more (16-32) channels.

6.3 Cache Inefficiency

Inspired by recent works on host network stacks [2, 34, 64] highlighting that memory contention and LLC misses can degrade performance in 10+GB/s networks, we leverage our Layer-1 memory profiler on GB200 to investigate potential memory inefficiency in the Scale-up domain. This is particularly important since Scale-up networks operate at much higher bandwidths (TB/s). Since Scale-up networks' memory accesses are cacheable in the LLC (§2.3), we focus on quantifying the cache efficiency of communication kernels, using our ideal cache performance model as a reference (Tab. 2).

We first study how the LLC hit rate changes with the number of channels and the network bandwidth (fabBw). Our results presented in Fig. 14 demonstrate that the near-optimal cache performance is achievable when the network is relatively slow (~ 200 GB/s) with 8 channels, which is consistent with our theoretical memory model. However, when more channels are used (≥ 16) and the network gets fast (≥ 350 GB/s), the cache efficiency collapses with both read and write hit rates dropping to $\sim 0\%$ (excluding the always-cached external write flow), which significantly underperforms our model.

This finding suggests that under high bandwidth in the Scale-up network, network buffers are aggressively evicted from the LLC by other memory traffic. To validate this hypothesis, we conduct another comparative study by varying the network buffer size. The rationale is that network buffers are used cyclically. Smaller buffers imply a shorter reuse distance and a better temporal locality, which should improve LLC hit rate if eviction is the root cause.

Fig. 15 shows the LLC hit rates under varying buffer sizes with the number of channels fixed at 32 (the empirically optimal value on GB200). Our results show that with smaller network buffers (≤ 2 MB), the cache performance approaches the ideal case ($\sim 80\%$ hit rate in writes and $\sim 60\%$ in reads on *AllGather*). In contrast, using the default (4MB) or larger network buffer sizes, the cache performance degrades significantly to the worst ($\sim 0\%$, excluding the always-cached external writes), confirming our hypothesis that network buffers are aggressively evicted due to other memory traffic. Even worse, in this regime, the LLC provides little benefit and may even hurt performance due to near-complete 100% replacement.

The analysis using our Layer-1 memory profiler highlights severe inefficiency ($\sim 100\%$ LLC misses) in the current network buffer design, the worst case from the host network perspective. We also identify that network buffer management is inherently complex, involving many coupled factors. For instance, while smaller buffers improve LLC performance, they also increase synchronization frequency, which can degrade performance from the network side.

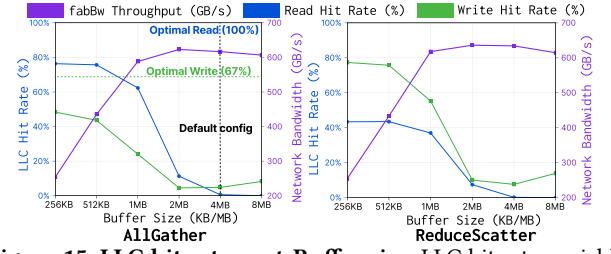


Figure 15: LLC hit rate w.r.t. Buffer size. LLC hit rates quickly drop to the worst ($\sim 0\%$) with larger (2/4/8MB) buffer sizes.

Table 8: NCCL fabBw and memory usage on GB200-NVL4, 2GB.

Case	fabBw	DRAM R	DRAM W	LLC R hit%	LLC W hit%
RS baseline	632.717	3.785G	2.003G	0.31%	7.78%
RS tuned	648.787	2.449G	1.976G	35.17%	8.40%
AG baseline	614.425	2.180G	3.603G	0.68%	4.85%
AG tuned	625.306	923.3M	3.580G	57.18%	4.86%

Moreover, managing caches and buffers is a hardware-software co-design problem. For instance, we conduct a case study on NCCL via the new LLC eviction priority hardware support (exposed as `.L2::evict_first/evict_last` in PTX [46]) that can tune the cache behavior for specific instructions and memory flows, e.g., `evict_first` reduces cache priority, making streaming data faster to be replaced with less contention with reusable, persistent data.

Our results in Tab. 8 highlight that by configuring streaming flows (①, ⑤) to be `evict_first`, while buffer stores (②, ③) and buffer loads (④) to be `evict_last` and `evict_first`² respectively, we can improve NCCL-optimized *ReduceScatter* fabric bandwidth by 16.07GB/s to 648.787GB/s³, where memory analysis contributes to a 34.86% increase in the LLC read hit rate. These speedups indicate that superior LLC behavior improves bandwidth, underscoring the deep interplay between the memory system and Scale-up networks.

7 Discussions and Future Directions

1d-oriented Scale-up Measurement Support: Though 1d semantics do not need a fence as the arrival confirmation and as probing hooks, fence is still effective to guard 1d data arrival. Therefore, by manually adding fence (with only $\sim 0.22\%$ overhead), FABRICPERF can measure latency/throughput of 1d semantics, and could be simpler as all timestamps are local without the need for GTPP.

Alternative Designs: Alternative approaches (e.g., adding hardware routines) are possible but face scalability issues because when lowering layers, packets are split into many small chunks for parallelism over lanes (similar to TCP $\rightarrow$ IP packet transition), and Layer-1/2 packets (e.g., Ethernet [27] at bytes-level) far exceed FABRICPERF's packets (Layer-3, split-by-fence at KB/MB level). This requires prohibitively large buffers/hardware, making FABRICPERF's Layer 3 design a better balance point between granularity and scalability, than the application, network, and physical layers.

New Scale-up Networks in the Future: There are new protocols, such as Scale-up Ethernet [6] and UALink [29] that will reintroduce an open-standard network layer (NIC-like Packet/Routing) to

²buffer loads (④) favor `evict_first` because NCCL uses circular buffers.

³This is below Tab. 5 since here we use NVL4 than NVL16 to fix CUPTI inter-node hangs for LLC metrics. On NVL16 the *ReduceScatter* speedup is 680.22GB/s $\rightarrow$ 690.8GB/s.

Scale-up networks, which may reinstate the network layer observability. Apart from the scalability issue mentioned above, we believe that FABRICPERF will continue to be instrumental in Scale-up network observability: ❶ Protocol layer has unique phenomena/problems, and network layer observability is orthogonal to FABRICPERF (Layer-1/3), just as IP profiling (traceroute/ping) does not disqualify TCP observability (RTT/cwnd). ❷ Even with an accessible network layer, observability would become accelerator-oriented rather than CPU/NIC-oriented because PCIe (100GB/s) is the fundamental bottleneck for Scale-up (TB/s). Thereby, FABRICPERF's designs, e.g., GPTP, would still play critical roles. ❸ Open Scale-up standards also bring customized, mutually incompatible protocols, e.g., Helios/Gaudi/Maia [1, 9, 28], making unified network-layer observability difficult. Thus, FABRICPERF, which captures memory instructions as the fundamental commonness, will continue to serve as a unified toolkit complementary to future open network layers.

Limitations and Future Directions: As a pioneering step toward understanding a new and underexplored networking paradigm, FABRICPERF's current limitations stem primarily from constrained observability and programmability in existing GPU-fabric stacks. For example, since the Layer-2 packet-processing engine is not programmable, FABRICPERF cannot directly profile at the network layer. As a result, we cannot observe routing decisions, packetization details, or Layer-2 latency. Currently, FABRICPERF also does not cover Scale-up switches, e.g., NvSwitch [31] and UASwitch [29], which run independently from GPUs and expose limited public interfaces for observability. Therefore, FABRICPERF can only infer switch effects indirectly through end-to-end latency.

Despite these limitations, FABRICPERF still enables solid measurements (§6) that reveal the incompleteness and inefficiencies in existing Scale-up network stacks. We hope FABRICPERF serves not only as a measurement toolkit to open the black-box Scale-up networks, but also as a call for future research and engineering towards more complete and efficient Scale-up network stacks.

8 Related Works

Scale-up Networks: As an emerging network domain, research on Scale-up networks is still in its early stage. One line of work focuses on designing open industry standards based on mature network protocols, such as the Scale-up Ethernet [6] and Ultra Accelerated Link (UALink) [29], which extends Ethernet's physical and network layers for the Scale-up domain. These open standards are still under development. We envision FABRICPERF, with its transport-layer support, can help close the observability gap needed to evaluate and refine these protocols.

Another line of work studies proprietary Scale-up networks (NvLink [42], InfiniteFabrics [56]). These efforts focus on the integration of Scale-up networks with computation kernels, e.g., designing better computation-communication overlapping [20, 60], and scheduling of Scale-up network traffic [8, 36, 39, 68], at the application layer. We envision FABRICPERF's packet-level measurements can provide deeper insights into the underlying network behavior.

Scale-up Network Software Profilers: Software profilers, e.g., the benchmarking suite `nccl-tests` [48] or the general PyTorch profiler [61], are the main source of today's understanding of Scale-up networks. These software profilers measure the network from

the host side via the communication kernel duration (Layer-4 latency) and infer fabric bandwidths via algorithm-based conversion. However, host-side profiling limits visibility into fine-grained, packet-level measurements, which FABRICPERF targets and enables. **GPU Hardware Profilers:** Hardware profilers, e.g., NVIDIA CUPTI [43] or AMD GPA [3], measure the (communication) kernel performance through sampling PMU, e.g., hit/miss counters at caches. Existing hardware profilers focus more on the computing pipeline and the memory system, with complete analysis from LSU utilization to stall cycles due to memory I/O. However, measurements on Scale-up fabrics remain incomplete. For example, CUPTI [43] only exposes limited counters on bytes received or transmitted through NvLink that can be normalized for throughput estimation, without other packet-level analysis. Moreover, hardware-based profilers are *inflexible* since all metrics are fixed by the PMU design, and new fabric metrics generally require additional modeling and inference, which motivates FABRICPERF's software-hardware co-design.

GPU Kernel Instrumentation: Dynamic instrumentation [7, 14, 57] that *adds code to running programs* has been proven effective for CPU-NIC network profiling, such as the success of eBPF [14] and XDP [10]. For GPUs, there are also some instrumentation tools for NVIDIA [5, 58, 63] or AMD [11] devices, and cross-platform solutions like Neutrino [25]. However, these tools primarily target compute kernels, e.g., GEMM, running on a single device, and do not address the unique challenges of communication kernels, including multi-GPU processing, rank identification and clock synchronization. Built upon these tools, FABRICPERF bridges this gap to enable transport-layer probing for Scale-up communication.

9 Conclusion

Scale-up GPU fabrics are becoming a critical part of modern ML networks, yet their behavior below the application layer remains poorly understood. We present FABRICPERF, a Scale-up network measurement toolkit that enables fine-grained packet-level timestamping and cross-layer hardware analysis, without NIC support. FABRICPERF achieves this with a distinctive software-hardware co-design that integrates transport-layer probing and memory-based traffic analysis. We implement FABRICPERF for the CUDA/NvLink ecosystem and validate its accuracy and efficiency via extensive evaluation. Our measurements using FABRICPERF through case studies on channel imbalance in the network domain and cache inefficiency in the memory domain reveal previously unnoticed behaviors and inefficiencies. We propose practical solutions, a work-stealing load balancer and an LLC eviction-priority tuning, to mitigate these bottlenecks and improve Scale-up network performance. We believe FABRICPERF takes a pioneering step towards more observable Scale-up networks, underpinning more complete and efficient Scale-up network stacks.

Ethical Statement: This work does not raise any ethical issues.

Acknowledgements

We sincerely thank the anonymous reviewers and our shepherd for their insightful feedback. We also thank Netpreme Corporation for providing the valuable GB200 NVL72 system for extensive evaluation and experiments. This work is supported by NSFC under grant No. 62222216 and Hong Kong RGC GRF No. 17211725.

References

- [1] Maia 200: The AI accelerator built for inference. 2026. <https://blogs.microsoft.com/blog/2026/01/26/maia-200-the-ai-accelerator-built-for-inference/>.
- [2] Saksham Agarwal, Arvind Krishnamurthy, and Rachit Agarwal. 2023. Host Congestion Control. In *Proceedings of the ACM SIGCOMM 2023 Conference* (New York, NY, USA) (ACM SIGCOMM '23). Association for Computing Machinery, New York, NY, USA, 275–287. doi:10.1145/3603269.3604878
- [3] AMD. 2024. GPU Performance API. <https://gpuopen.com/gpuperfapi/>
- [4] IEEE Standard Association. 2002. IEEE 1588-2002: Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems. <https://standards.ieee.org/ieee/1588/3140/>
- [5] Lorenz Braun and Holger Fröning. 2019. CUDA Flux: A Lightweight Instruction Profiler for CUDA Applications. In *2019 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*. Association for Computing Machinery, New York, NY, USA, 73–81. doi:10.1109/PMBS49563.2019.00014
- [6] Broadcom. 2025. Scale-Up Ethernet Framework Specification. <https://docs.broadcom.com/doc/scale-up-ethernet-framework>
- [7] Derek Bruening, Timothy Garnett, and Saman Amarasinghe. 2003. An infrastructure for adaptive dynamic optimization. In *the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization* (San Francisco, California, USA) (CGO '03). Association for Computing Machinery, USA, 265–275.
- [8] Jiamin Cao, Yu Guan, Kun Qian, Jiaqi Gao, Wencong Xiao, Jianbo Dong, Binzhang Fu, Dennis Cai, and Ennan Zhai. 2024. Crux: GPU-Efficient Communication Scheduling for Deep Learning Training. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney, NSW, Australia) (ACM SIGCOMM '24). Association for Computing Machinery, New York, NY, USA, 1–15. doi:10.1145/3651890.3672239
- [9] Intel Gaudi Network Configuration. 2026. https://docs.habana.ai/en/latest/Management_and_Monitoring/Network_Configuration/index.html.
- [10] Jonathan Corbet. 2018. Accelerating networking with AF_XDP. <https://lwn.net/Articles/750845/>
- [11] Sébastien Darche and Michel R. Dagenais. 2024. Low-Overhead Trace Collection and Profiling on GPU Compute Kernels. *ACM Trans. Parallel Comput.* 11, 2, Article 9 (June 2024), 24 pages. doi:10.1145/3649510
- [12] DeepSeek. 2025. DeepEP: an efficient expert-parallel communication library. <https://github.com/deepseek-ai/DeepEP>
- [13] Open Fabrics Enterprise Distribution. 2025. Performance Test for RDMA. <https://github.com/linux-rdma/perftest>
- [14] eBPF. 2024. Extended Berkeley Packet Filter. <https://ebpf.io>
- [15] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. arXiv:2101.03961 [cs.LG] <https://arxiv.org/abs/2101.03961>
- [16] Linux Foundation. 2025. Data plane development kit. <https://www.dpdk.org/>
- [17] Network Time Foundation. 2024. The Linux PTP Project. <https://linuxptp.sourceforge.net>
- [18] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuaiqiang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Hongyi Zeng. 2024. RDMA over Ethernet for Distributed Training at Meta Scale. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney, NSW, Australia) (ACM SIGCOMM '24). Association for Computing Machinery, New York, NY, USA, 57–70. doi:10.1145/3651890.3672233
- [19] Ehab Ghabashneh, Yimeng Zhao, Cristian Lumezanu, Neil Spring, Srikanth Sundaresan, and Sanjay Rao. 2022. A microscopic view of bursts, buffer contention, and loss in data centers. In *Proceedings of the 22nd ACM Internet Measurement Conference* (Nice, France) (IMC '22). Association for Computing Machinery, New York, NY, USA, 567–580. doi:10.1145/3517745.3561430
- [20] Yue Guan, Xinwei Qiang, Zaifeng Pan, Daniels Johnson, Yuanwei Fang, Keren Zhou, Yuke Wang, Wanlu Li, Yufei Ding, and Adnan Aziz. 2025. Mercury: Unlocking Multi-GPU Operator Optimization for LLMs via Remote Memory Scheduling. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles* (Lotte Hotel World, Seoul, Republic of Korea) (SOSP '25). Association for Computing Machinery, New York, NY, USA, 1046–1061. doi:10.1145/3731569.3764798
- [21] Chuanxiong Guo, Haitao Wu, Zhong Deng, Gaurav Soni, Jianxi Ye, Jitu Padhye, and Marina Lipshteyn. 2016. RDMA over Commodity Ethernet at Scale. In *Proceedings of the 2016 ACM SIGCOMM Conference* (Florianopolis, Brazil) (SIGCOMM '16). Association for Computing Machinery, New York, NY, USA, 202–215. doi:10.1145/2934872.2934908
- [22] Chuanxiong Guo, Lihua Yuan, Dong Xiang, Yingnong Dang, Ray Huang, Dave Maltz, Zhaoyi Liu, Vin Wang, Bin Pang, Hua Chen, Zhi-Wei Lin, and Varugis Kurien. 2015. Pingmesh: A Large-Scale System for Data Center Network Latency Measurement and Analysis. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication* (London, United Kingdom) (SIGCOMM '15). Association for Computing Machinery, New York, NY, USA, 139–152. doi:10.1145/2785956.2787496
- [23] Christian Hopps. 2000. Analysis of an Equal-Cost Multi-Path Algorithm. <https://datatracker.ietf.org/doc/html/rfc2992>
- [24] Zhiyi Hu, Siyuan Shen, Tommaso Bonato, Sylvain Jeaugey, Cedell Alexander, Eric Spada, James Dinan, Jeff Hammond, and Torsten Hoefler. 2025. Demystifying NCCL: An In-depth Analysis of GPU Communication Protocols and Algorithms. arXiv:2507.04786 [cs.DC] <https://arxiv.org/abs/2507.04786>
- [25] Songlin Huang and Chenshu Wu. 2025. Neutrino: Fine-grained GPU Kernel Profiling via Programmable Probing. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. USENIX Association, Boston, MA, 331–355. <https://www.usenix.org/conference/osdi25/presentation/huang-songlin>
- [26] IEEE. 2011. 802.1Qbb – Priority-based Flow Control. <https://1.ieee802.org/dcb/802-1qbb/>
- [27] IEEE. 2022. 802.3-2022 - IEEE Standard for Ethernet. <https://ieeexplore.ieee.org/document/9844436>
- [28] AMD "Helios": Advancing Openness in AI Infrastructure. 2026. <https://www.amd.com/en/blogs/2025/amd-helios-ai-rack-built-on-metas-2025-ocp-design.html>.
- [29] Nathan Kalyanasundharam. 2025. Introducing UALink 200G 1.0 Specification. https://ualinkconsortium.org/wp-content/uploads/2025/04/UALink-1.0-White-Paper_FINAL.pdf
- [30] Gautam Kumar, Nandita Dukkkipati, Keon Jang, Hassan M. G. Wassel, Xian Wu, Behnam Montazeri, Yaogong Wang, Kevin Springborn, Christopher Alfeld, Michael Ryan, David Wetherall, and Amin Vahdat. 2020. Swift: Delay is Simple and Effective for Congestion Control in the Datacenter. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication* (Virtual Event, USA) (SIGCOMM '20). Association for Computing Machinery, New York, NY, USA, 514–528. doi:10.1145/3387514.3406591
- [31] Ang Li, Shuaiwen Leon Song, Jieyang Chen, Jiajia Li, Xu Liu, Nathan R. Tallent, and Kevin J. Barker. 2020. Evaluating Modern GPU Interconnect: PCIe, NVLink, NV-SLI, NVSwitch and GPUDirect. *IEEE Transactions on Parallel and Distributed Systems* 31, 1 (Jan. 2020), 94–110. doi:10.1109/tpds.2019.2928289
- [32] Yuliang Li, Rui Miao, Hongqiang Harry Liu, Yan Zhuang, Fei Feng, Lingbo Tang, Zheng Cao, Ming Zhang, Frank Kelly, Mohammad Alizadeh, and Minlan Yu. 2019. HPCC: high precision congestion control. In *Proceedings of the ACM Special Interest Group on Data Communication* (Beijing, China) (SIGCOMM '19). Association for Computing Machinery, New York, NY, USA, 44–58. doi:10.1145/3341302.3342085
- [33] GNU Linux. 2025. adjtime - correct the time to synchronize the system clock. <https://man7.org/linux/man-pages/man3/adjtime.3.html>
- [34] Bowen Liu, Xinyang Huang, Qijing Li, Zhuobin Huang, Yijun Sun, Wenxue Li, Junxue Zhang, Ping Yin, and Kai Chen. 2025. CEO: A Cache-Efficient Network I/O Architecture for NIC-CPU Data Paths. In *Proceedings of the ACM SIGCOMM 2025 Conference* (São Francisco Convent, Coimbra, Portugal) (SIGCOMM '25). Association for Computing Machinery, New York, NY, USA, 381–394. doi:10.1145/3718958.3750488
- [35] Kefei Liu, Zhuo Jiang, Jiao Zhang, Haoran Wei, Xiaolong Zhong, Lizhuang Tan, Tian Pan, and Tao Huang. 2023. Hostping: Diagnosing Intra-host Network Bottlenecks in RDMA Servers. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, 15–29. <https://www.usenix.org/conference/nsdi23/presentation/liu-kefei>
- [36] Tongrui Liu, Chenyang Hei, Fuliang Li, Chengxi Gao, Jiamin Cao, Tianshu Wang, Ennan Zhai, and Xingwei Wang. 2025. ResCCL: Resource-Efficient Scheduling for Collective Communication. In *Proceedings of the ACM SIGCOMM 2025 Conference* (São Francisco Convent, Coimbra, Portugal) (SIGCOMM '25). Association for Computing Machinery, New York, NY, USA, 55–70. doi:10.1145/3718958.3750514
- [37] Jie Lu, Jiaqi Gao, Fei Feng, Zhiqiang He, Menglei Zheng, Kun Liu, Jun He, Binbin Liao, Suwei Xu, Ke Sun, Yongjia Mo, Qinghua Peng, Jilie Luo, Qingxu Li, Gang Lu, Zishu Wang, Jianbo Dong, Kunling He, Sheng Cheng, Jiamin Cao, Hairong Jiao, Pengcheng Zhang, Shu Ma, Lingjun Zhu, Chao Shi, Yangming Zhang, Yiquan Chen, Wei Wang, Shuhong Zhu, Xingru Li, Qiang Wang, Jiang Liu, Chao Wang, Wei Lin, Ennan Zhai, Jiesheng Wu, Qiang Liu, Binzhang Fu, and Dennis Cai. 2025. Alibaba Stellar: A New Generation RDMA Network for Cloud AI. In *Proceedings of the ACM SIGCOMM 2025 Conference* (São Francisco Convent, Coimbra, Portugal) (SIGCOMM '25). Association for Computing Machinery, New York, NY, USA, 453–466. doi:10.1145/3718958.3750539
- [38] Martin Lévesque and David Tipper. 2016. A Survey of Clock Synchronization Over Packet-Switched Networks. *IEEE Communications Surveys & Tutorials* 18, 4 (2016), 2926–2947. doi:10.1109/COMST.2016.2590438
- [39] Kshiteej Mahajan, Ching-Hsiang Chu, Srinivas Sridharan, and Aditya Akella. 2023. Better Together: Jointly Optimizing ML Collective Scheduling and Execution Planning using SYNDICATE. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, 809–824. <https://www.usenix.org/conference/nsdi23/presentation/mahajan>
- [40] Pooria Namyar, Yuliang Li, Weitao Wang, Nandita Dukkkipati, Kk Yap, Junzhi Gong, Chen Chen, Peixuan Gao, Devdeep Ray, Gautam Kumar, Yidan Ma, Ramesh

- Govindan, and Amin Vahdat. 2025. Firefly: Scalable, Ultra-Accurate Clock Synchronization for Datacenters. In *Proceedings of the ACM SIGCOMM 2025 Conference* (São Francisco Convent, Coimbra, Portugal) (SIGCOMM '25). Association for Computing Machinery, New York, NY, USA, 434–452. doi:10.1145/3718958.3750502
- [41] NCCL. 2025. User Buffer Registration. <https://docs.nvidia.com/deeplearning/ncl/user-guide/docs/usage/bufferreg.html>
- [42] NVIDIA. 2014. NVIDIA® NVLink™ High-Speed Interconnect: Application Performance. <https://info.nvidia.com/rs/nvidia/images/NVIDIA%20NVLink%20High-Speed%20Interconnect%20Application%20Performance%20Brief.pdf>
- [43] NVIDIA. 2024. CUDA Profiling Tools Interface. <https://docs.nvidia.com/cupti/index.html>
- [44] NVIDIA. 2024. NCCL: Optimized primitives for interGPU communication. <https://github.com/NVIDIA/nccl>
- [45] NVIDIA. 2024. NSight Compute System. <https://developer.nvidia.com/nsight-compute>
- [46] NVIDIA. 2024. PTX ISA 8.5. <https://docs.nvidia.com/cuda/parallel-thread-execution>
- [47] NVIDIA. 2025. GB200 NVL72: Powering the era of accelerated computing. <https://www.nvidia.com/en-us/data-center/gb200-nvl72/>
- [48] NVIDIA. 2025. NCCL Tests. <https://github.com/NVIDIA/nccl-tests>
- [49] NVIDIA. 2025. NVIDIA BlueField Platform. <https://www.nvidia.com/en-us/networking/products/data-processing-unit/>
- [50] Collective Operations. 2026. <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/collectives.html>
- [51] Pitch Patarasuk and Xin Yuan. 2009. Bandwidth optimal all-reduce algorithms for clusters of workstations. *J. Parallel Distrib. Comput.* 69, 2 (Feb. 2009), 117–124. doi:10.1016/j.jpdc.2008.09.002
- [52] Boris Pismenny, Adam Morrison, and Dan Tsafir. 2023. ShRing: Networking with Shared Receive Rings. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. USENIX Association, Boston, MA, 949–968. <https://www.usenix.org/conference/osdi23/presentation/pismenny>
- [53] Chenchen Qi, Wenfei Wu, Yongcan Wang, Keqiang He, Yu-Hsiang (Sean) Kao, Zongying He, Chen-Yu Yen, Zhuo Jiang, Feng Luo, Surendra Anubolu, Yanjin Gao, Bingfeng Lin, Wenda Ni, Yiming Yang, Donglin Wei, Boyang Zhou, Jian Wang, and Shan Ding. 2025. SGLB: Scalable and Robust Global Load Balancing in Commodity AI Clusters. In *Proceedings of the ACM SIGCOMM 2025 Conference* (São Francisco Convent, Coimbra, Portugal) (SIGCOMM '25). Association for Computing Machinery, New York, NY, USA, 626–644. doi:10.1145/3718958.3750527
- [54] Kun Qian, Yongqing Xi, Jiamin Cao, Jiaqi Gao, Yichi Xu, Yu Guan, Binzhang Fu, Xuemei Shi, Fangbo Zhu, Rui Miao, Chao Wang, Peng Wang, Pengcheng Zhang, Xianlong Zeng, Eddie Ruan, Zhiping Yao, Ennan Zhai, and Dennis Cai. 2024. Alibaba HPN: A Data Center Network for Large Language Model Training. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney, NSW, Australia) (ACM SIGCOMM '24). Association for Computing Machinery, New York, NY, USA, 691–706. doi:10.1145/3651890.3672265
- [55] Sudarsanan Rajasekaran, Sanjoli Narang, Anton A. Zabreyko, and Manya Ghobadi. 2024. MLTCP: A Distributed Technique to Approximate Centralized Flow Scheduling For Machine Learning. In *Proceedings of the 23rd ACM Workshop on Hot Topics in Networks* (Irvine, CA, USA) (HotNets '24). Association for Computing Machinery, New York, NY, USA, 167–176. doi:10.1145/3696348.3696878
- [56] Gabin Schieffer, Ruimin Shi, Stefano Markidis, Andreas Herten, Jennifer Faj, and Ivy Peng. 2024. Understanding Data Movement in AMD Multi-GPU Systems with Infinity Fabric. arXiv:2410.00801 [cs.DC] <https://arxiv.org/abs/2410.00801>
- [57] Julian Seward and Nicholas Nethercote. 2005. Using Valgrind to Detect Undefined Value Errors with Bit-Precision. In *2005 USENIX Annual Technical Conference (ATC '05)*. USENIX Association, Anaheim, CA, 17–30. <https://www.usenix.org/conference/2005-usenix-annual-technical-conference/using-valgrind-detect-undefined-value-errors-bit>
- [58] Du Shen, Shuaiwen Leon Song, Ang Li, and Xu Liu. 2018. CUDAAdvisor: LLVM-based runtime profiling for modern GPUs. In *the 2018 International Symposium on Code Generation and Optimization* (Vienna, Austria) (CGO '18). Association for Computing Machinery, New York, NY, USA, 214–227. doi:10.1145/3168831
- [59] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2020. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv:1909.08053 [cs.CL] <https://arxiv.org/abs/1909.08053>
- [60] Stuart H. Sul, Simran Arora, Benjamin F. Spector, and Christopher Ré. 2025. ParallelKittens: Systematic and Practical Simplification of Multi-GPU AI Kernels. arXiv:2511.13940 [cs.DC] <https://arxiv.org/abs/2511.13940>
- [61] PyTorch Team. 2024. PyTorch Profiler. https://pytorch.org/tutorials/recipes/recipes/profiler_recipe.html
- [62] Mellanox Technologies. 2003. Introduction to InfiniBand. https://network.nvidia.com/pdf/whitepapers/IB_Intro_WP_190.pdf
- [63] Oreste Villa, Mark Stephenson, David Nellans, and Stephen W. Keckler. 2019. NVBit: A Dynamic Binary Instrumentation Framework for NVIDIA GPUs. In *the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) (MICRO '19). Association for Computing Machinery, New York, NY, USA, 372–383. doi:10.1145/3352460.3358307
- [64] Midhul Vuppapapati, Saksham Agarwal, Henry Schuh, Baris Kasikci, Arvind Krishnamurthy, and Rachit Agarwal. 2024. Understanding the Host Network. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney, NSW, Australia) (ACM SIGCOMM '24). Association for Computing Machinery, New York, NY, USA, 581–594. doi:10.1145/3651890.3672271
- [65] Xinchun Wan, Hong Zhang, Hao Wang, Shuihai Hu, Junxue Zhang, and Kai Chen. 2020. RAT - Resilient Allreduce Tree for Distributed Machine Learning. In *Proceedings of the 4th Asia-Pacific Workshop on Networking* (Seoul, Republic of Korea) (APNet '20). Association for Computing Machinery, New York, NY, USA, 52–57. doi:10.1145/3411029.3411037
- [66] Hao Wang, Han Tian, Jingrong Chen, Xinchun Wan, Jiacheng Xia, Gaoxiong Zeng, Wei Bai, Junchen Jiang, Yong Wang, and Kai Chen. 2024. Towards Domain-Specific Network Transport for Distributed DNN Training. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, Santa Clara, CA, 1421–1443. <https://www.usenix.org/conference/nsdi24/presentation/wang-hao>
- [67] Yibo Zhu, Haggai Eran, Daniel Firestone, Chuanxiong Guo, Marina Lipshteyn, Yehonatan Liron, Jitendra Padhye, Shachar Raindel, Mohamad Haj Yahia, and Ming Zhang. 2015. Congestion Control for Large-Scale RDMA Deployments. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication* (London, United Kingdom) (SIGCOMM '15). Association for Computing Machinery, New York, NY, USA, 523–536. doi:10.1145/2785956.2787484
- [68] Siyuan Zhuang, Zhuohan Li, Danyang Zhuo, Stephanie Wang, Eric Liang, Robert Nishihara, Philipp Moritz, and Ion Stoica. 2021. Hoplite: efficient and fault-tolerant collective communication for task-based distributed systems. In *Proceedings of the ACM SIGCOMM 2021 Conference* (Virtual Event, USA) (SIGCOMM '21). Association for Computing Machinery, New York, NY, USA, 641–656. doi:10.1145/3452296.3472897

Appendices

Appendices are supporting material that has not been peer-reviewed.

A Group Network Model

This is the group communication model used in Ring/Tree algorithms [51, 65], which can be composed, asymmetrically, by our basic building blocks (Fig. 6).

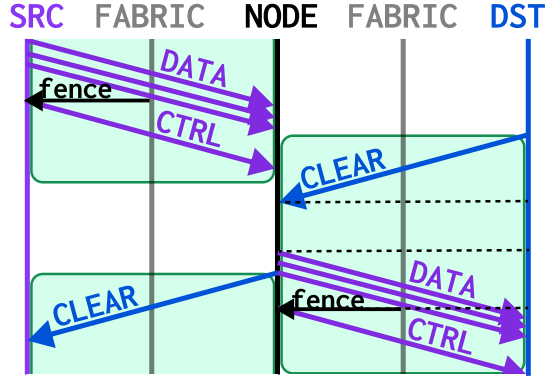


Figure 16: Chain network model in Ring/Tree Group Communication. Can be decomposed into three basic models (green boxes).

B Latency Distribution

We identify that the high P95 (purple line) and P99 (red line) latency start to diverge from P50 latency (blue line) from 12 channels where networks are far from utilization. 24 channels is the setup of Fig. 12.

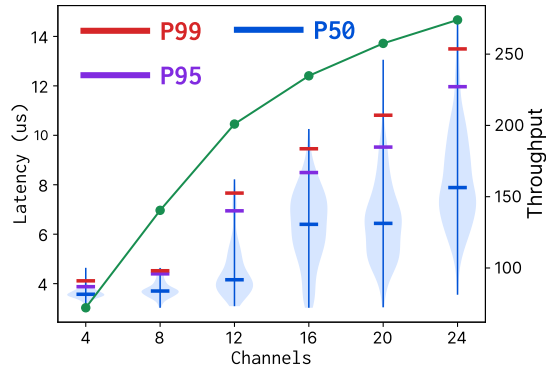


Figure 17: Latency Distribution versus No. Channels